

レッスン04

シリアルポートの使用

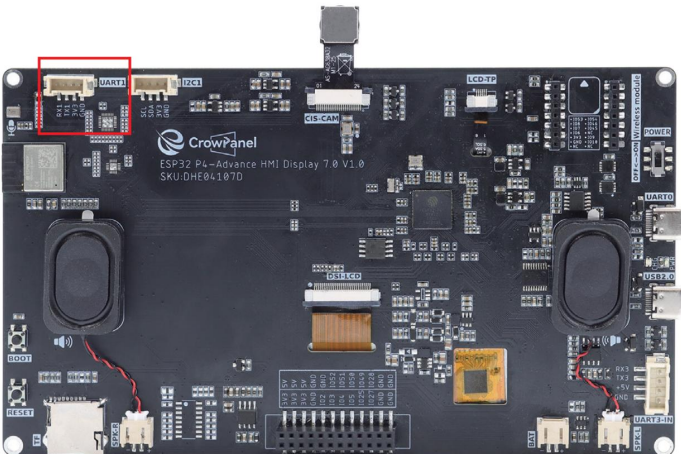
導入

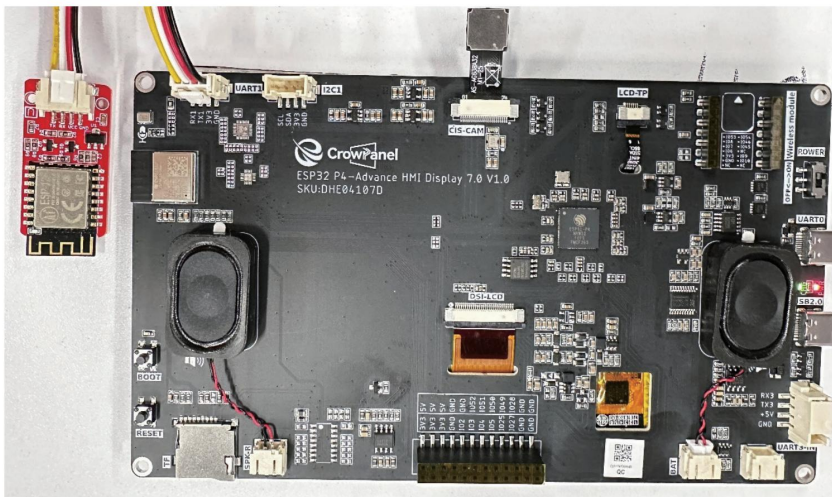
この授業では、シリアルポートコンポーネントの使い方を解説していきます。Advance-P4のUART1インターフェースを介して、Wi-Fiシリアルモジュールと通信を行います。

Advance-P4はシリアルポートを介してWi-Fiモジュールに接続します。Wi-FiモジュールにATコマンドを送信した後、Wi-FiモジュールがWi-Fiネットワークに接続できるようにします。

このレッスンで使用するハードウェア

Advance-P4のUART1インターフェース





動作効果図

コードを実行すると、ESP-IDFのモニターに送信したATコマンドと、シリアルポート経由でWi-Fiモジュールから返された応答が表示されます。(緑色はAdvance-P4からの送信、白色はWi-Fiモジュールからの応答を表します。)

```

C main.c  C bsp_uart.c  C bsp_uart.h
main > C main.c @ wifitaskvoid
72 void wifitask(void *arg)
73     for (int i = 0; i < 5; i++)
74     {
75         if (connect_wifi())
76         {
77             vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
78         }
79     }
80     if (!connected)
81     {
82         ESP_LOGE(TAG, "Cannot connect to WiFi, stopping task"); // Log failure after all attempts
83         vTaskDelete(NULL); // Delete task if connection failed
84     }
85     // Get IP address of the module
86     send_at_command("AT+CIFSR", pdMS_TO_TICKS(1000));
87     // Enable multiple connections mode
88     send_at_command("AT+CIPMUX=1", pdMS_TO_TICKS(1000));
89     // Start TCP server on port 80
90     send_at_command("AT+CIPSERVER=1,80", pdMS_TO_TICKS(1000));
91     while (1)
92     {
93         // TODO: Can read UART data here to process TCP requests
94         vTaskDelay(pdMS_TO_TICKS(1000)); // Delay to reduce CPU usage
95     }
96
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF
I [16085] WEPI AT: AT Response: AT+QIDP="elecrow888","elecrow888"
WEPI DISCONNECT
WEPI CONNECTED
WEPI GOT IP
OK
I [16085] WEPI AT: WEPI Connected
I [17995] WEPI AT: AT Response: AT+CIFSR
+CIFSR:APIP,"192.168.4.1"
+CIFSR:APIP,"207.171.171.204:80"
+CIFSR:STAPIP,"192.168.4.133"
+CIFSR:STAPIP,"3c:71:b1:f4:d4:79"
OK

```

重要な説明

このクラスの主な焦点は、シリアルポートの使い方です。ここでは、`bsp_uart` という新しいコンポーネントを皆さんに提供します。このコンポーネントは主に、シリアルポートの初期化、シリアルポートの設定、および関連するインターフェースの使用に使用されます。ご存知のとおり、作成したインターフェースは適切なタイミングで呼び出すことができます。

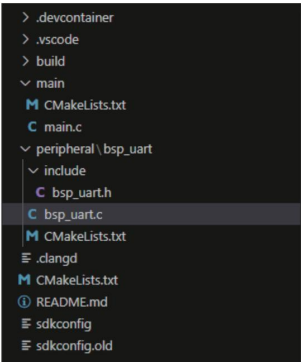
次に、`bsp_uart` コンポーネントの理解に焦点を当てます。

まず、下記のGitHubリンクをクリックして、このレッスンのコードをダウンロードしてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson04-Serial_port_usage

次に、このレッスンのコードをVS Codeにドラッグして、プロジェクトファイルを開きます。

• ファイルを開くと、このプロジェクトのフレームワークが表示されます。



このクラスの例では、「peripheral」ディレクトリの下に「bsp_uart」という名前の新しいフォルダが作成されました。

「bsp_uart」フォルダ内に、新しい「include」フォルダと「CMakeLists.txt」ファイルが作成されました。

「bsp_uart」フォルダには「bsp_uart.c」ドライバファイルが含まれており、「include」フォルダには「bsp_uart.h」ヘッダーファイルが含まれています。

「CMakeLists.txt」ファイルは、ドライバをビルドシステムに統合し、プロジェクトが「bsp_uart.c」に記述されたシリアル通信機能を利用できるようにします。

シリアルポート通信コード

- シリアルポート通信のドライバコードは、2つのファイルで構成されています: `"bsp_uart.c"` および `"bsp_uart.h"`
- 次に、まず「bsp_uart.h」プログラムを解析します。
- `"bsp_uart.h"` はシリアルポート通信のヘッダーファイルで、主に以下の目的で使用されます。
 - 外部向けに「bsp_uart.c」で実装されている関数、マクロ、変数を宣言します。
使用するプログラム
 - 他の .c ファイルからこのモジュールを呼び出すには、単に「`#include "bsp_uart.h"`」を含めるだけで済みます。
- 言い換えれば、それはインターフェース層であり、どの関数や定数を公開できるかを示します。
モジュールの内部詳細を隠蔽しつつ、外部で使用される。

- このコンポーネントでは、使用する必要のあるすべてのライブラリが「bsp_uart.h」ファイルに配置され、一元管理されています。

```

3
4  /*-----Header file declaration-----*/
5  #include <string.h>           // Standard C library for string manipulation
6  #include <stdint.h>          // Standard integer type definitions (e.g., uint8_t, int32_t)
7  #include "freertos/FreeRTOS.h" // FreeRTOS core definitions
8  #include "freertos/task.h"    // FreeRTOS task management APIs
9  #include "esp_log.h"          // ESP-IDF logging library
10 #include "esp_err.h"          // ESP-IDF error codes
11 #include "driver/uart.h"       // ESP-IDF UART driver APIs
12 /*-----Header file declaration end-----*/
13

```

- 次に、使用する必要のある変数の宣言と関数の宣言を行います。これらの関数の具体的な実装は「bsp_uart.c」にあります。

- 呼び出しや管理を容易にするため、これらはすべて「bsp_uart.h」に統一的に配置されています。

(「bsp_uart.c」で使用されている箇所を見れば、それぞれの機能が理解できるでしょう。)

```

14 /*-----Variable declaration-----*/
15 #define UART_TAG "UART"       // Logging tag for UART module
16 #define UART_INFO(fmt, ...) ESP_LOGI(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART info log
17 #define UART_DEBUG(fmt, ...) ESP_LOGD(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART debug log
18 #define UART_ERROR(fmt, ...) ESP_LOGE(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART error log
19
20 #define UART_IN_EXTRA_GPIO_TXD 34 // Define GPIO number 34 as UART TXD pin for input extra UART
21 #define UART_IN_EXTRA_GPIO_RXD 33 // Define GPIO number 33 as UART RXD pin for input extra UART
22
23 #define UART1_EXTRA_GPIO_TXD 47 // Define GPIO number 47 as UART1 TXD pin
24 #define UART1_EXTRA_GPIO_RXD 48 // Define GPIO number 48 as UART1 RXD pin
25
26 typedef enum
27 {
28     UART_SCAN = 1,           // UART state: scanning or waiting for input
29     UART_DECODE,            // UART state: decoding received data
30     UART_ERR,                // UART state: error occurred
31 } uart_state;
32
33 int SendData(const char *data); // Function to send data over UART
34 esp_err_t uart_init();          // Function to initialize UART
35 uart_state get_uart_status();    // Function to get current UART state
36 void set_uart_status(uart_state status); // Function to set UART state
37
38 /*-----Variable declaration end-----*/
39 #endif                          // End of header guard
40

```

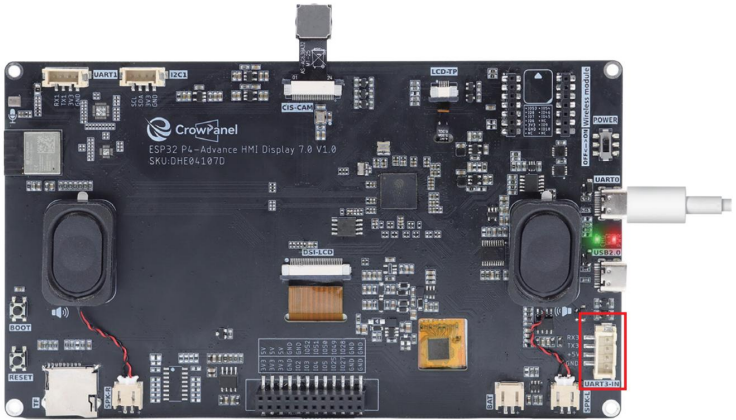
- ここにはシリアルポートのピンが2セットあることがわかります。最初のセットは UART_IN で、図に示すように UART3-IN インターフェースの TX ピンと RX ピンです。(このレッスンでは使用していません。これらのピンは、今後の使用を容易にするために用意しました。)

ただし、このインターフェースは外部電源を供給できないことに注意してください。

```

20 #define UART_IN_EXTRA_GPIO_TXD 34 // Define GPIO number 34 as UART TXD pin for input extra UART
21 #define UART_IN_EXTRA_GPIO_RXD 33 // Define GPIO number 33 as UART RXD pin for input extra UART
22

```



- もう一つのグループは、この授業で使用するUART1インターフェースです。前述したように、このインターフェースは通常のGPIOポートとしてだけでなく、シリアルポートとしても使用できます。この授業では、このインターフェースを使用します。

```
23 #define UART1_EXTRA_GPIO_TXD 47 // Define GPIO number 47 as UART1 TXD pin
24 #define UART1_EXTRA_GPIO_RXD 48 // Define GPIO number 48 as UART1 RXD pin
```

- もう一度「bsp_uart.c」を見て、各関数が具体的に何をするのかを確認してみましょう。
- bsp_uart: bsp_uart コンポーネントは ESP32 UART ハードウェアをカプセル化し、初期化、データ送信、受信、およびステータス管理のための統一されたインターフェースを提供し、基盤となるドライバの詳細を隠蔽することで、上位レイヤーのタスク（WiFi AT制御タスクなど）がUARTを介して外部デバイスと安定かつ確実に通信できるようにします。

次に、画面表示を実装するために呼び出すインターフェースとして、以下の関数を使用します。

uart_init():

- この関数は、ESP32-P4 の UART2 を初期化し、その構成を行う役割を担います。通信パラメータには、ボーレート、データビット、ストップビット、パリティビット、フロー制御モードなどが含まれます。また、UARTドライバをインストールし、TX/RXピンを指定します。
- 基盤となる uart_driver_install()、uart_param_config()、および uart_set_pin() はハードウェアの詳細を隠蔽し、上位レイヤーのタスクが UART 初期化の面倒な操作を気にしなくて済むようにします。

- この関数を呼び出すと、UART ハードウェアが準備完了となり、データ処理を実行できるようになります。送受信。通常、システム起動時、または通信が必要になる前に呼び出されます。
- Advance-P4には、UART0、UART1、UART2、UART3、UART4、UART4、UART4、UART5、UART6、UART7、UART8 およびUART3-IN。
- UART0 は、電源供給とコードアップロード用のデフォルトインターフェースです。デフォルトでは、UART_NUM_0。
- すると、UART_NUM_1とUART_NUM_2が残ります。
- ここでは、シリアルポートインターフェースを1つしか使用しないため、どちらのポートでも自由に選択できます。そこで、私は UART_NUM_2を選択します。
- UART3-INインターフェースも使用する場合は、バインドするポート番号とピンが一致しており、競合していないことを確認してください。

```

13 int SendData(const char *data)           // Function to send a string of data through UART2
14 {
15     const int len = strlen(data);        // Get the length of the input string
16     const int txBytes = uart_write_bytes(UART_NUM_2, data, len); // Write string to UART2
17     return txBytes;                       // Return number of bytes actually sent
18 }
19
20 esp_err_t uart_init()                   // Function to initialize UART2
21 {
22     esp_err_t err = ESP_OK;              // Variable to store error status, default to ESP_OK
23     const uart_config_t uart_config = {   // UART configuration structure
24         .baud_rate = 115200,              // Set baud rate to 115200
25         .data_bits = UART_DATA_8_BITS,    // 8 data bits per frame
26         .parity = UART_PARITY_DISABLE,    // Disable parity check
27         .stop_bits = UART_STOP_BITS_1,    // 1 stop bit
28         .flow_ctrl = UART_HW_FLOWCTRL_DISABLE, // Disable hardware flow control
29         .source_clk = UART_SCLK_DEFAULT,  // Use default UART clock source
30     };
31
32     err = uart_driver_install(UART_NUM_2, 1024 * 2, 0, 0, NULL, 0); // Install UART2 driver with RX buffer size 2048 bytes
33     if (err != ESP_OK)                  // Check if driver installation failed
34     {
35         UART_ERROR("extra uart driver install fail"); // Log error if installation failed
36         return err;                       // Return error code
37     }
38     uart_set_pin(UART_NUM_2, UART1_EXTRA_GPIO_TXD, UART1_EXTRA_GPIO_RXD, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
39     // Configure UART2 TX and RX pins, keep RTS/CTS unchanged
40     err = uart_param_config(UART_NUM_2, &uart_config); // Apply UART parameter configuration
41     if (err != ESP_OK)                  // Check if configuration failed
42     {
43         return err;                       // Return error code
44     }
45     return ESP_OK;                      // Return success if everything is OK

```

SendData(const char *data):

- この関数は、文字列データを UART2 に送信するために使用されます。まず文字列の長さを計算し、次に uart_write_bytes() を呼び出してデータを UART ハードウェアに送信します。この関数は実際に送信されたバイト数を返すため、上位レイヤーが送信が成功したかどうかを判断するのに役立ちます。また、基盤となるドライバインターフェースをカプセル化することで、上位レベルのタスクやモジュールが、毎回バッファやバイト長を処理することなく、SendData() を呼び出すだけでコマンドやデータを安全に送信できるようにします。

```

13 int SendData(const char *data) // Function to send a string of data through UART2
14 {
15     const int len = strlen(data); // Get the length of the input string
16     const int txBytes = uart_write_bytes(UART_NUM_2, data, len); // Write string to UART2
17     return txBytes; // Return number of bytes actually sent
18 }

```

C main.c C bsp_uart.c **C bsp_uart.h** X

```

peripheral > bsp_uart > include > C bsp_uart.h > ...
9 #include "esp_log.h" // ESP-IDF logging library
10 #include "esp_err.h" // ESP-IDF error codes
11 #include "driver/uart.h" // ESP-IDF UART driver APIs
12 /*-----Header file declaration end-----*/
13
14 /*-----Variable declaration-----*/
15 #define UART_TAG "UART" // Logging tag for UART module
16 #define UART_INFO(fmt, ...) ESP_LOGI(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART info log
17 #define UART_DEBUG(fmt, ...) ESP_LOGD(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART debug log
18 #define UART_ERROR(fmt, ...) ESP_LOGE(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART error log
19
20 #define UART_IN_EXTRA_GPIO_TXD 34 // Define GPIO number 34 as UART TXD pin for input extra UART
21 #define UART_IN_EXTRA_GPIO_RXD 33 // Define GPIO number 33 as UART RXD pin for input extra UART
22
23 #define UART1_EXTRA_GPIO_TXD 47 // Define GPIO number 47 as UART1 TXD pin
24 #define UART1_EXTRA_GPIO_RXD 48 // Define GPIO number 48 as UART1 RXD pin
25
26 typedef enum
27 {
28     UART_SCAN = 1, // UART state: scanning or waiting for input
29     UART_DECODE, // UART state: decoding received data
30     UART_ERR, // UART state: error occurred
31 } uart_state;
32
33 int SendData(const char *data); // Function to send data over UART
34 esp_err_t uart_init(); // Function to initialize UART
35
36 /*-----Variable declaration end-----*/
37 #endif // End of header guard

```

- 以上がbsp_uartのコンポーネントの説明です。呼び出し方を必ず確認してください。
これらのインターフェース。
- 次に、呼び出しを行う必要がある場合は、次の場所にある「CMakeLists.txt」ファイルも構成する必要があります。
「bsp_uart」フォルダ内。
- このファイルは「bsp_uart」フォルダに配置され、その主な機能は、ESP-IDFのビルドシステム（CMake）に「bsp_uart」コンポーネントをコンパイルおよび登録する方法を通知することです。

EXPLORER ... C main.c C bsp_uart.c C bsp_uart.h **M CMakeLists.txt** X

LESSON04

- > .devcontainer
- > .vscode
- > build
- main
 - M CMakeLists.txt
 - C main.c
- peripheral\bsp_uart
 - include
 - C bsp_uart.h
 - C bsp_uart.c
 - M CMakeLists.txt**
 - clangd
 - M CMakeLists.txt

```

peripheral > bsp_uart > M CMakeLists.txt
1 FILE(GLOB_RECURSE component_sources "*.c")
2
3 idf_component_register(SRCS ${component_sources}
4 | | | | | INCLUDE_DIRS "include"
5 | | | | | REQUIRES driver)
6
7
8

```


- これが「ドライバ」と呼ばれる理由は、「bsp_uart.h」ファイル内で呼び出しているためです（システムライブラリである他のライブラリについては、何も追加する必要はありません）。

```

1  #ifndef _BSP_UART_H_           // Prevent multiple inclusion of this header file
2  #define _BSP_UART_H_
3
4  /*-----Header file declaration-----*/
5  #include <string.h>             // Standard C library for string manipulation
6  #include <stdint.h>             // Standard integer type definitions (e.g., uint8_t, int32_t)
7  #include "freertos/FreeRTOS.h" // FreeRTOS core definitions
8  #include "freertos/task.h"     // FreeRTOS task management APIs
9  #include "esp_log.h"           // ESP-IDF logging library
10 #include "esp_err.h"           // ESP-IDF error codes
11 #include "driver/uart.h"       // ESP-IDF UART driver APIs
12 /*-----Header file declaration end-----*/
13
14 /*-----Variable declaration-----*/
15 #define UART_TAG "UART"        // Logging tag for UART module
16 #define UART_INFO(fmt, ...) ESP_LOGI(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART info log
17 #define UART_DEBUG(fmt, ...) ESP_LOGD(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART debug log
18 #define UART_ERROR(fmt, ...) ESP_LOGE(UART_TAG, fmt, ##_VA_ARGS_) // Macro for UART error log
19
20 #define UART_IN_EXTRA_GPIO_TXD 34 // Define GPIO number 34 as UART TXD pin for input extra UART
21 #define UART_IN_EXTRA_GPIO_RXD 33 // Define GPIO number 33 as UART RXD pin for input extra UART
22
23 #define UART1_EXTRA_GPIO_TXD 47 // Define GPIO number 47 as UART1 TXD pin
24 #define UART1_EXTRA_GPIO_RXD 48 // Define GPIO number 48 as UART1 RXD pin

```

主な機能

- メインフォルダはプログラム実行のコアディレクトリであり、main関数の実行ファイルmain.c。
- ビルドシステムの「CMakeLists.txt」ファイルにメインフォルダを追加します。

```

1  #include "bsp_uart.h"
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "string.h"
5  #include "esp_log.h"
6
7  #define WIFI_SSID "elecrow888" // WiFi network name
8  #define WIFI_PASS "elecrow2014" // WiFi network password
9  #define AT_RESPONSE_MAX_LEN 512 // Maximum length for AT command responses
10
11 static const char *TAG = "WIFI_AT"; // tag for logging messages
12
13 /* Read UART return data */
14 static int uart_read_response(char *buffer, size_t len, TickType_t timeout)
15 {
16     int total = 0; // Total number of bytes read
17     int read_bytes = 0; // Bytes read in current iteration
18     TickType_t start = xTaskGetTickCount(); // Get current system tick count
19     // Continue reading until timeout or buffer is full
20     while ((xTaskGetTickCount() - start) < timeout && total < len - 1)
21     {
22         // Read bytes from UART2 with 20ms timeout per read
23         read_bytes = uart_read_bytes(UART_NUM_2, (uint8_t *)buffer + total, len - total - 1, portTICK_PERIOD_MS);
24         if (read_bytes > 0)
25         {
26             total += read_bytes; // Accumulate total bytes read
27         }
28     }
29     buffer[total] = '\0'; // Null-terminate the response string
30     return total; // Return total bytes read
31 }
32
33 /* Send AT command and wait for OK response */
34 static bool send_at_command(const char *cmd, TickType_t timeout)
35 {
36     char response[AT_RESPONSE_MAX] = {0}; // Buffer to store response
37     SendData(cmd); // Send the AT command
38     SendData("\r\n"); // Send command terminator
39     uart_read_response(response, AT_RESPONSE_MAX, timeout); // Read response

```

- これはアプリケーション全体のエントリー ファイルです。ESP-IDF には int main() はありませんが、プログラムは void app_main(void) から実行されます。

- ESP-IDFフレームワークでは、`app_main()`はプログラム全体のメインエントリーポイントです。
標準C言語の`main()`関数に相当します。
- ESP32-P4の電源がオンまたは再起動されると、システムは`app_main()`を実行して起動します。
ユーザーのタスクとアプリケーションロジック。
- `main.c`について説明しましょう
- 機能: `bsp_uart` コンポーネントのインターフェースを呼び出し、FreeRTOS が
スケジューラを使用して `wifi_task` を実行し、AT コマンドを送信して `wifi` モジュールを制御します。
Wi-Fiに接続してください。

"bsp_uart.h":

このファイルは、カスタムUARTカプセル化コンポーネント「`bsp_uart`」をインポートし、UART初期化、データ送信、データ受信、ステータス管理などのインターフェースを提供することで、上位レイヤーのタスクがUARTを介して外部デバイスと容易に通信できるようにします。

`#include "freertos/FreeRTOS.h":`

このファイルは、FreeRTOSカーネルの基本ヘッダーファイルをインポートし、タスクスケジューリング、時間管理、セマフォ、キューなど、FreeRTOSを使用するために必要な基本的な操作関数と型定義を提供します。

`#include "freertos/task.h":`

このファイルは、FreeRTOSにおけるタスク管理に関連するインターフェースをインポートします。これには、タスクを作成するための`xTaskCreate()`、タスクの遅延を行うための`vTaskDelay()`、タスクを削除するための`vTaskDelete()`などの関数が含まれており、マルチタスクのスケジューリングと管理に使用されます。

`#include "string.h":`

このファイルは、文字列の長さの計算、部分文字列の検索、文字列の書式設定などの操作を行うために、C標準ライブラリの文字列処理関数 (`strlen()`、`strstr()`、`snprintf()`など)をインポートします。

`#include "esp_log.h":`

このファイルは、デバッグ情報、エラー情報、システムステータスを出力するために使用される、ESP-IDFが提供するログシステムインターフェースをインポートします。`ESP_LOGI()`、`ESP_LOGE()`、`ESP_LOGD()`などの関数を提供します。

```
main > C main.c > ...
1  #include "bsp_uart.h"
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "string.h"
5  #include "esp_log.h"
```

- WiFiの名前 (SSID)が定義されました。これは、モジュールが指定されたWiFiネットワークに接続できるようにするためのATコマンドを構築するためにプログラムで使用されます。
- WiFiのパスワード (Password)も定義されており、SSIDとともに、WiFiネットワークに接続するためのATコマンドで使用されます。

```
7 #define WIFI_SSID "elecrow888" // WiFi network name
8 #define WIFI_PASS "elecrow2014" // WiFi network password
```

- ATを受信するバッファの最大長を表す定数を定義する
コマンド応答のサイズは512バイトです。これにより、受信データが境界を超えないことが保証されます。
- 静的文字列をログタグ (Tag) として定義します。これは、次のようなログ関数で使用されます。
ESP_LOGI() と ESP_LOGE() は、異なるモジュールの出力を区別し、デバッグと問題箇所の特定を容易にします。

```
10 #define AT_RESPONSE_MAX 512 // Maximum length for AT command responses
11 static const char *TAG = "WIFI_AT"; // Tag for logging messages
```

uart_read_response(char *buffer, size_t len, TickType_t timeout):

- 「uart_read_response()」関数は、bsp_uartコンポーネントの中核となる関数で、UARTからのデータ受信を担当します。この関数は、ESP32の「uart_read_bytes()」インターフェースを繰り返し呼び出し、UART2で受信したデータをユーザーが指定したバッファに格納します。また、タイムアウト制御もサポートしています。
- この関数は、読み取りのたびに実際に受信したバイト数を蓄積し、バッファの末尾に0を追加することで、返されるデータが有効なC文字列であることを保証します。バッファオーバーフローを防ぐだけでなく、指定された時間内にデータが返されるまで継続的に待機するため、ATコマンド応答や外部デバイスから返されるその他のデータの読み取りに適しています。これにより、上位レベルのタスクは、基盤となるUARTドライバを直接操作することなく、受信データを安全かつ確実に取得できます。

```
13 /* Read UART return data */
14 static int uart_read_response(char *buffer, size_t len, TickType_t timeout)
15 {
16     int total = 0; // Total number of bytes read
17     int read_bytes = 0; // Bytes read in current iteration
18     TickType_t start = xTaskGetTickCount(); // Get current system tick count
19     // Continue reading until timeout or buffer is full
20     while ((xTaskGetTickCount() - start) < timeout && total < len - 1)
21     {
22         // Read bytes from UART2 with 20ms timeout per read
23         read_bytes = uart_read_bytes(UART_NUM_2, (uint8_t *) (buffer + total), len - total - 1, 20 / portTICK_PERIOD_MS);
24         if (read_bytes > 0)
25         {
26             total += read_bytes; // Accumulate total bytes read
27         }
28     }
29     buffer[total] = '\0'; // Null-terminate the response string
30     return total; // Return total bytes read
31 }
32
```

send_at_command(const char *cmd, TickType_t timeout):

- "send_at_command()" 関数は、AT モジュールにコマンドを送信し、応答を待つために使用される、"bsp_uart" コンポーネント内の高レベル ラッパー関数です。
- まず、ユーザーが渡したAT命令を「SendData()」関数を使用してUARTに送信し、次にコマンド終端文字としてキャリッジリターンとラインフィード文字を送信します。その後、「uart_read_response()」を呼び出してモジュールから返されたデータを読み取り、バッファに保存すると同時に、デバッグ目的でログを出力します。
- この関数は、返された文字列に「OK」が含まれているかどうかを確認します。含まれている場合は、コマンドの実行が成功した場合は「true」が返され、そうでない場合はコマンドの失敗を示す「false」が返されます。

```
I (25585) WIFI_AT: AT Response: AT+CIPMUX=1
OK
I (26585) WIFI_AT: AT Response: AT+CIPSERVER=1,80
OK
```

- この関数は、送信、受信、結果の完全なプロセスをカプセル化します。判断機能により、上位レベルのタスクは、基盤となるUARTの読み書きや応答解析の詳細を扱うことなく、単一のインターフェースを介してATモジュールを安全かつ簡単に操作できるようになります。

```
33  /* Send AT command and wait for OK response */
34  static bool send_at_command(const char *cmd, TickType_t timeout)
35  {
36      char response[AT_RESPONSE_MAX] = {0}; // Buffer to store response
37      SendData(cmd); // Send the AT command
38      SendData("\r\n"); // Send command terminator
39
40      uart_read_response(response, AT_RESPONSE_MAX, timeout); // Read response
41      ESP_LOGI(TAG, "AT Response: %s", response); // Log the response
42
43      // Check if response contains "OK"
44      if (strstr(response, "OK") != NULL)
45          return true; // Command succeeded
46      else
47          return false; // Command failed
48  }
```

connect_wifi():

- 「connect_wifi()」関数は、ESP32をATモジュールを介して指定されたWiFiネットワークに接続する。
- まず、WiFiに接続するためのATコマンド「AT+CWMODE=1, 'SSID」を作成します。
128バイトのバッファに「PASSWORD」を格納し、WiFi名への接続が試行されていることを示すログメッセージを出力します。
- 次に、「send_at_command()」関数を呼び出してコマンドを送信し、タイムアウトを5秒に設定してモジュールの応答を待ちます。
- この関数は、接続が成功したかどうかを、
応答結果 : 応答が「OK」の場合は、「WiFi接続済み」ログを出力してtrueを返します。接続が成功しなかった場合は、エラーログを出力してfalseを返します。
- この関数は、AT コマンドの構築から始まるプロセス全体をカプセル化します。
接続結果を判定するコマンドを送信することで、上位レベルのタスクがそれを直接呼び出し、基盤となるUARTやコマンド解析の詳細を処理することなくWiFi接続を実現できるようにする。

```

50  /* WiFi connection function */
51  static bool connect_wifi()
52  {
53      char cmd[128]; // Buffer to build AT command
54
55      // Construct AT command to join WiFi network
56      snprintf(cmd, sizeof(cmd), "AT+CWMODE=1,\"%s\",\"%s\"", WIFI_SSID, WIFI_PASS);
57      ESP_LOGI(TAG, "Connecting to Wifi: %s", WIFI_SSID); // Log connection attempt
58
59      // Send command with 5 second timeout and return result
60      if (send_at_command(cmd, pdMS_TO_TICKS(5000)))
61      {
62          ESP_LOGI(TAG, "WiFi Connected"); // Log successful connection
63          return true;
64      }
65      else
66      {
67          ESP_LOGE(TAG, "Failed to connect Wifi"); // Log connection failure
68          return false;
69      }
70  }

```

wifi_task(void *arg):

- この関数は、先に説明したすべてのインターフェースを呼び出します。
- 関数 wifi_task() は AT WiFi モジュールと通信する FreeRTOS タスクです。
UART を介して WiFi 接続と TCP サーバーの初期化を実現する。
- このタスクはまず UART を初期化します。初期化に失敗した場合は、システムの安定性を確保するために自身を削除します。

```

72 void wifi_task(void *arg)
73 {
74     // Initialize UART communication
75     if (uart_init() != ESP_OK)
76     {
77         ESP_LOGE(TAG, "UART init failed"); // Log UART initialization failure
78         vTaskDelete(NULL); // Delete current task if initialization fails
79         return;
80     }
81
82     // Configure module to AP+STA mode (Access Point + Station)
83     send_at_command("AT+CMODE=3", pdMS_TO_TICKS(1000));
84     // Reset the module to apply settings
85     send_at_command("AT+RST", pdMS_TO_TICKS(2000));
86     vTaskDelay(pdMS_TO_TICKS(3000)); // Delay to allow module to restart
87
88     // Attempt to connect to WiFi, maximum 5 tries
89     bool connected = false;
90     for (int i = 0; i < 5; i++)
91     {
92         if (connect_wifi())
93         {
94             connected = true; // Mark as connected if successful
95             break;
96         }
97         vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
98     }
99
100     if (!connected)
101     {
102         ESP_LOGE(TAG, "Cannot connect to WiFi, stopping task"); // Log failure after all attempts
103         vTaskDelete(NULL); // Delete task if connection failed
104     }
105
106     // Get IP address of the module
107     send_at_command("AT+CIFSR", pdMS_TO_TICKS(1000));
108     // Enable multiple connections mode
109     send_at_command("AT+CIPMUX=1", pdMS_TO_TICKS(1000));
110     // Start TCP server on port 80
111     send_at_command("AT+CIPSERVER=1,80", pdMS_TO_TICKS(1000));
112
113     while (1)
114     {
115         // TODO: Can read UART data here to process TCP requests
116         vTaskDelay(pdMS_TO_TICKS(1000)); // Delay to reduce CPU usage
117     }

```

- 次に、モジュールをAP + STAモードに設定し、リセットして設定を反映させます。
効果。

```

72 void wifi_task(void *arg)
73 {
74     // Initialize UART communication
75     if (uart_init() != ESP_OK)
76     {
77         ESP_LOGE(TAG, "UART init failed"); // Log UART initialization failure
78         vTaskDelete(NULL); // Delete current task if initialization fails
79         return;
80     }
81
82     // Configure module to AP+STA mode (Access Point + Station)
83     send_at_command("AT+CMODE=3", pdMS_TO_TICKS(1000));
84     // Reset the module to apply settings
85     send_at_command("AT+RST", pdMS_TO_TICKS(2000));
86     vTaskDelay(pdMS_TO_TICKS(3000)); // Delay to allow module to restart
87
88     // Attempt to connect to Wifi, maximum 5 tries
89     bool connected = false;
90     for (int i = 0; i < 5; i++)
91     {
92         if (connect_wifi())
93         {
94             connected = true; // Mark as connected if successful
95             break;
96         }
97         vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
98     }
99

```

- その後、プロセスは指定されたWiFiへの接続を最大5回繰り返し試行します。

接続に失敗するたびに2秒間の遅延が発生します。最終的に接続が確立できない場合は、エラーメッセージが表示され、タスクは削除されます。

```

72 void wifi_task(void *arg)
73 {
74     // Initialize UART communication
75     if (uart_init() != ESP_OK)
76     {
77         ESP_LOGE(TAG, "UART init failed"); // Log UART initialization failure
78         vTaskDelete(NULL); // Delete current task if initialization fails
79         return;
80     }
81
82     // Configure module to AP+STA mode (Access Point + Station)
83     send_at_command("AT+CMODE=3", pdMS_TO_TICKS(1000));
84     // Reset the module to apply settings
85     send_at_command("AT+RST", pdMS_TO_TICKS(2000));
86     vTaskDelay(pdMS_TO_TICKS(3000)); // Delay to allow module to restart
87
88     // Attempt to connect to Wifi, maximum 5 tries
89     bool connected = false;
90     for (int i = 0; i < 5; i++)
91     {
92         if (connect_wifi())
93         {
94             connected = true; // Mark as connected if successful
95             break;
96         }
97         vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
98     }
99
100     if (!connected)
101     {
102         ESP_LOGE(TAG, "Cannot connect to Wifi, stopping task"); // Log failure after all attempts
103         vTaskDelete(NULL); // Delete task if connection failed
104     }
105

```

- 接続が成功すると、モジュールのIPアドレスを取得し、マルチ接続モードを有効にし、TCPサーバーを起動してポート80でリスンさせます。

```

82 // Configure module to AP+STA mode (Access Point + Station)
83 send_at_command("AT+CMODE=3", pdMS_TO_TICKS(1000));
84 // Reset the module to apply settings
85 send_at_command("AT+RST", pdMS_TO_TICKS(2000));
86 vTaskDelay(pdMS_TO_TICKS(3000)); // Delay to allow module to restart
87
88 // Attempt to connect to Wifi, maximum 5 tries
89 bool connected = false;
90 for (int i = 0; i < 5; i++)
91 {
92     if (connect_wifi())
93     {
94         connected = true; // Mark as connected if successful
95         break;
96     }
97     vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
98 }
99
100 if (!connected)
101 {
102     ESP_LOGE(TAG, "Cannot connect to Wifi, stopping task"); // Log failure after all attempts
103     vTaskDelete(NULL); // Delete task if connection failed
104 }
105
106 // Get IP address of the module
107 send_at_command("AT+CIFSR", pdMS_TO_TICKS(1000));
108 // Enable multiple connections mode
109 send_at_command("AT+CIPMUX=1", pdMS_TO_TICKS(1000));
110 // Start TCP server on port 80
111 send_at_command("AT+CIPSERVER=1,80", pdMS_TO_TICKS(1000));
112
113 while (1)
114 {
115     // TODO: Can read UART data here to process TCP requests
116     vTaskDelay(pdMS_TO_TICKS(1000)); // Delay to reduce CPU usage
117 }
118 }

```

- 最後に、無限ループに入り、TCPリクエストの後続処理のためにインターフェースを保持し、遅延によってCPU使用率を削減することで、Wifiネットワーク管理およびサービスの全プロセスを完了します。

```

100 if (!connected)
101 {
102     ESP_LOGE(TAG, "Cannot connect to Wifi, stopping task"); // Log failure after all attempts
103     vTaskDelete(NULL); // Delete task if connection failed
104 }
105
106 // Get IP address of the module
107 send_at_command("AT+CIFSR", pdMS_TO_TICKS(1000));
108 // Enable multiple connections mode
109 send_at_command("AT+CIPMUX=1", pdMS_TO_TICKS(1000));
110 // Start TCP server on port 80
111 send_at_command("AT+CIPSERVER=1,80", pdMS_TO_TICKS(1000));
112
113 while (1)
114 {
115     // TODO: Can read UART data here to process TCP requests
116     vTaskDelay(pdMS_TO_TICKS(1000)); // Delay to reduce CPU usage
117 }
118 }

```


•次に、メイン関数であるapp_mainが登場します。

•app_main() は、ESP-IDF プログラムのエントリ関数であり、標準 C プログラムの main() 関数に相当します。このコードでは、その役割は非常に明確です。xTaskCreate() を呼び出して、「wifi_task」という名前の FreeRTOS タスクを作成します。タスク関数は wifi_task であり、スタック領域を 4096 バイト割り当て、優先度を 5 に設定し、タスクパラメータを渡さず、タスクハンドルを NULL に設定します (タスクハンドルを保存しません)。

•このコード行の核心的な意味は、WiFiの初期化とTCPサーバーのロジックを、FreeRTOSスケジューラの管理下で実行される独立したタスクにカプセル化することです。これにより、メインプログラムのエントリポイントをシンプルに保ちながら、WiFi接続タスクを他のタスクをブロックすることなく並列実行できます。

```
120 void app_main(void)
121 {
122     // Create Wifi task with 4096 bytes stack, priority 5
123     xTaskCreate(wifi_task, "wifi_task", 4096, NULL, 5, NULL);
124 }
```

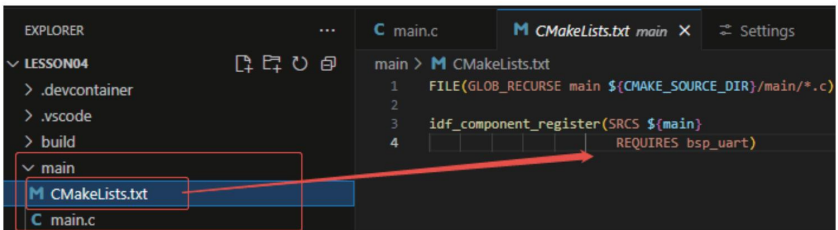
•それでは、「main」ディレクトリにある「CMakeLists.txt」ファイルを見てみましょう。

•このCMake設定の機能は以下のとおりです。

•すべての .c ソースファイルを「main/」ディレクトリに収集し、成分;

•「メイン」コンポーネントをESP-IDFビルドシステムに登録し、カスタムコンポーネント「bsp_uart」に依存します。

•このように、ビルドプロセス中に、ESP-IDF は最初に「bsp_uart」をビルドし、次に「main」をビルドします。



注 :以降のコースでは、新しい「CMakeLists.txt」ファイルをゼロから作成するのではなく、既存のファイルに若干の変更を加えて、他のドライバをメイン関数に統合していきます。

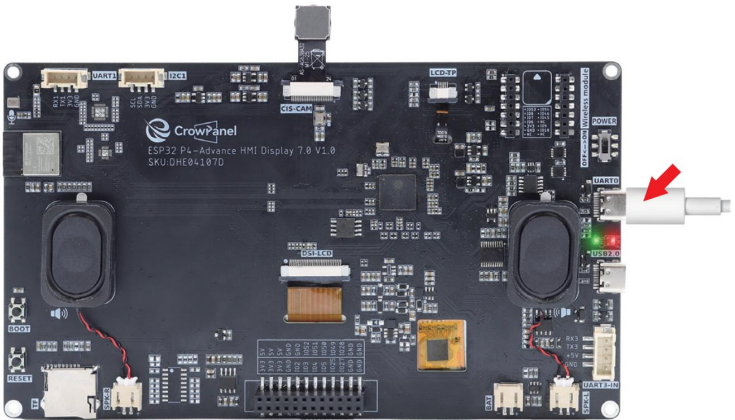
完全なコード

完全なコード実装をご覧になるには、下記のリンクをクリックしてください。

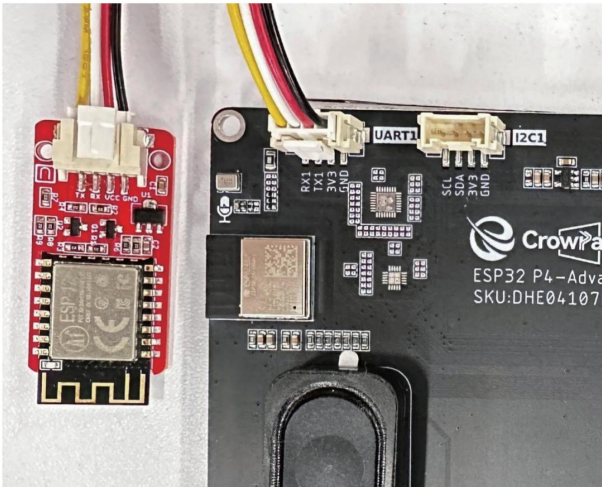
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson04-Serial_port_usage

プログラミング手順

- これでコードの準備ができました。次に、ESP32-P4 にフラッシュして、結果。
- まず、USBケーブルを使ってAdvance-P4デバイスをホストコンピュータに接続します。



- 次に、ESP8266 Wi-FiモジュールをUART1インターフェースに接続します。
- (UART1インターフェースのVCCをWi-FiモジュールのVCCピンに接続してください)
- (UART1インターフェースのGNDをWi-FiモジュールのGNDピンに接続してください)
- (UART1インターフェースのTXをWi-FiモジュールのRXピンに接続する) (クロス接続)
- (UART1インターフェースのRXをWi-FiモジュールのTXピンに接続する) (クロス接続)



- 書き込みプロセスを開始する前に、コンパイルされたすべてのファイルを削除し、プロジェクトを初期の「コンパイルされていない」状態に戻します。(これにより、その後のコンパイルが以前の操作の影響を受けないことが保証されます。)

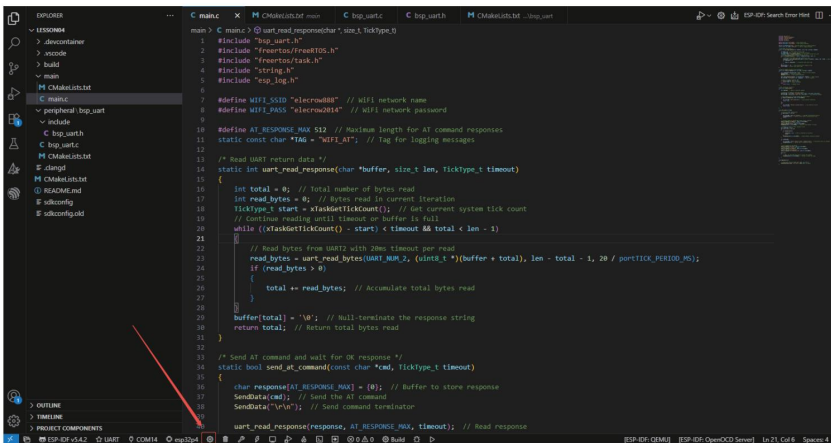
```

EXPLORER
...
C main.c x M CMakeLists.txt main C bsp_uart.c C bsp_uart.h M CMakeLists.txt ... bsp_uart

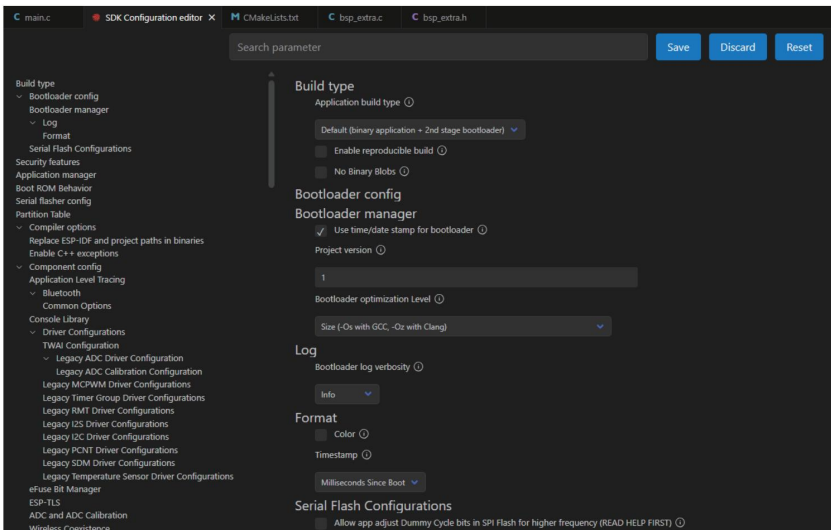
main > C main.c > @ uart_read_response(char*, size_t, TickType_t)
1 #include "bsp_uart.h"
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "string.h"
5 #include "esp_log.h"
6
7 #define WIFI_SSID "elecrow88" // WiFi network name
8 #define WIFI_PASS "elecrow2014" // WiFi network password
9
10 #define AT_RESPONSE_MAX 512 // Maximum length for AT command responses
11 static const char *TAG = "WIFI_AT"; // Tag for logging messages
12
13 /* Read UART return data */
14 static int uart_read_response(char *buffer, size_t len, TickType_t timeout)
15 {
16     int total = 0; // Total number of bytes read
17     int read_bytes = 0; // Bytes read in current iteration
18     TickType_t start = xTaskGetTickCount(); // Get current system tick count
19     // Continue reading until timeout or buffer is full
20     while ((xTaskGetTickCount() - start) < timeout && total < len - 1)
21     {
22         // Read bytes from UART2 with 20ms timeout per read
23         read_bytes = uart_read_bytes(UART_NUM_2, (uint8_t *) (buffer + total), len - total - 1, 20 / portTICK_PERIOD_MS);
24         if (read_bytes > 0)
25         {
26             total += read_bytes; // Accumulate total bytes read
27         }
28     }
29     buffer[total] = '\0'; // Null-terminate the response string
30     return total; // Return total bytes read
31 }
32
33 /* Send AT command and wait for OK response */
34 static bool send_at_command(const char *cmd, TickType_t timeout)
35 {
36     char response[AT_RESPONSE_MAX] = {0}; // Buffer to store response
37     SendData(cmd); // Send the AT command
38     SendData("\r\n"); // Send command terminator
39
40     uart_read_response(response, AT_RESPONSE_MAX, timeout); // Read response

```

- ここでは、最初のセクションの手順に従って、まずESP-IDFのバージョン、コードのアップロード方法、シリアルポート、および使用するチップを選択します。
- 次に、SDKを設定する必要があります。
- 下の画像にあるアイコンをクリックしてください。

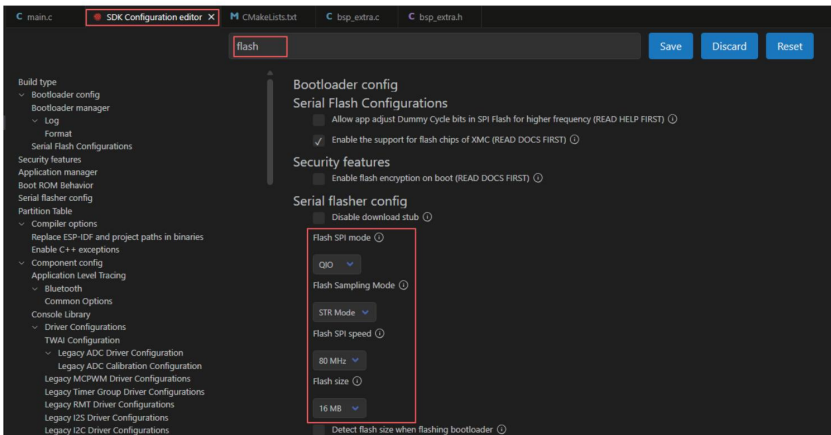


- 読み込み処理が完了するまでしばらくお待ちください。その後、関連するSDKの設定に進むことができます。



- 次に、検索ボックスに「flash」と入力して検索します。（フラッシュの設定が同じであることを確認してください）

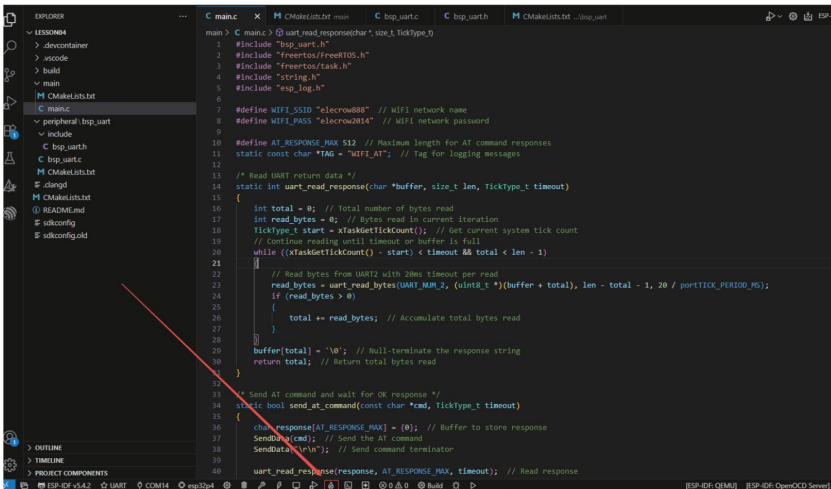
私のものと同じように。）



- 設定が完了したら、必ず設定を保存してください。

- その後、コードをコンパイルして書き込みます（これは最初のステップで詳しく説明しました）。
クラス）

- ここで、もう一つ非常に便利な機能をご紹介します。ボタンを1回押すだけで、コンパイル、アップロード、モニターの起動といった一連の作業を一度に実行できます。（ただし、コード全体にエラーがないことが前提となります。）



- しばらく待つと、コードのコンパイルとアップロードが完了し、モニターも開きました。
- コードを書き込んだ後、ESP-IDFのモニターを通して送信したATコマンドと、シリアルポート経由でWi-Fiモジュールから返された応答を確認できます。(緑色はAdvance-P4からの送信、白色はWi-Fiモジュールからの応答です。)

The screenshot shows the ESP-IDF IDE with a C source file open. The code defines a `wifi_task` that attempts to connect to a Wi-Fi network. It includes a delay between connection attempts and logs the status. The terminal output shows the successful connection process, including the AT response and the IP address assigned to the module.

```

C main.c  X  C bsp_uart.c  C bsp_uart.h
main> C main.c @ wifi_task(void)
72 void wifi_task(void *arg)
90 for (int i = 0; i < 5; i++)
92 if (connect_wifi())
94 {
95     vTaskDelay(pdMS_TO_TICKS(2000)); // Delay between connection attempts
96 }
97
98 if (!connected)
99 {
100     ESP_LOGE(TAG, "Cannot connect to WiFi, stopping task"); // Log failure after all attempts
101     vTaskDelete(NULL); // Delete task if connection failed
102 }
103
104 // Get IP address of the module
105 send_at_command("AT+CIFSR", pdMS_TO_TICKS(1000));
106 // Enable multiple connections mode
107 send_at_command("AT+CIPMUX=1", pdMS_TO_TICKS(1000));
108 // Start TCP server on port 80
109 send_at_command("AT+CIPSERVER=1,80", pdMS_TO_TICKS(1000));
110 while (1)
111 {
112     // TODO: Can read UART data here to process TCP requests
113     vTaskDelay(pdMS_TO_TICKS(1000)); // Delay to reduce CPU usage
114 }
115
116
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF
I (16585) WiFi AT: AT Response: AT+CNQAP="elecrow888","elecrow0814"
WiFi DISCONNECT
WiFi CONNECTED
WiFi Got IP
OK
I (16585) WiFi AT: WiFi Connected
I (17595) WiFi AT: AT Response: AT+CIFSR
+CIFSR:APIP,"192.168.4.1"
+CIFSR:APIP,"3e:71:bf:2d:fd:79"
+CIFSR:STAIP,"192.168.50.133"
+CIFSR:STAPIP,"3c:71:bf:2d:fd:79"
OK

```